DATABASE MANAGEMENT SYSTEMS (DBMS)

MiniProject / PBLCaseStudyFile

MOVIE TICKET BOOKING SYSTEM

Developed using MySQL and MySQL Workbench

Submitted by

Detail	Information
Student Name	Sania Amjad
UID	25BCD10080
Course / Branch	BCD
Semester	3rd
Subject	DatabaseManagement Systems
Submitted to	Mrs,Harkamal

GitHub Repository: https://github.com/_____/movie-ticket-booking-system

INDEX

Sr. No.	Topic	Page No.
1	Introduction	3
2	Problem Statement, Objectives and Scope	3
3	PBL / Case Study: Cinema Booking Management	5
4	Modules, Tools and Technologies Used	6
5	Theoretical Background (DBMS Concepts Used)	8
6	Database Design (ER Diagram and Data Dictionary)	10
7	Implementation: Database, Tables and Sample Data	12
8	SQL Queries with Output Screenshots	15
9	Final Verification	19
10	Results and Discussion	20
11	Limitations and Future Scope	20
12	Conclusion	21
13	Viva Preparation: Questions and Answers	22
14	References and GitHub Repository	22

Note: Figures 2 to 25 are the SQL statements and result grids from MySQL Workbench. All outputs are verified against the sample data given in this file.

1. INTRODUCTION

The Movie Ticket Booking System is a database management project developed using MySQL. It is designed to maintain and manage movie information, customer details, show schedules, and ticket bookings in one organised place.

In a cinema, managing movie schedules and booking records manually can be time-consuming and may lead to errors or duplicate records. This system provides an organised way to store, retrieve, update, and manage the information using a relational database.

The project consists of four main tables: Movies, Customers, Shows, and Bookings. These tables are connected using primary keys and foreign keys to maintain relationships and data integrity. For example, a booking cannot be stored for a customer or a show that does not exist. The system also allows users to view booking details, calculate ticket amounts, and generate simple reports using SQL queries. The total amount of a booking is not stored but calculated as the number of tickets multiplied by the ticket price, which avoids storing redundant data.

This project demonstrates the practical application of database concepts such as table creation, constraints, data manipulation, joins, and aggregate functions.

1.1 Why a Database for a Cinema?

A cinema deals with a large number of small, related facts every day: which film is running, at what time, at what price, who bought how many tickets, and how much money was collected. A database management system (DBMS) keeps all these facts in a structured form so that they can be searched and combined quickly. Instead of searching through registers, a single SQL query can show every booking of a customer or the total revenue of a day.

1.2 Overview of the Project

Item	Description
Project Title	Movie Ticket Booking System
Database Name	movie_ticket_booking
DBMS	MySQL (Relational Database Management System)
Interface	MySQL Workbench
Tables	Movies, Customers, Shows, Bookings
Type	Academic mini project (console-based database project, no front end)

2. PROBLEM STATEMENT, OBJECTIVES AND SCOPE

2.1 Problem Statement

Manual management of movie details, show schedules, customer information, and ticket bookings can result in data duplication, calculation errors, and difficulty retrieving records. Therefore, a database system is required to store this information in a structured and reliable manner.

2.2 Objectives

1. To maintain records of movies, including their titles, genres, and durations.
2. To store customer details in a structured database.

3.To manage movie show dates, timings, and ticket prices.
4.To record ticket bookings and the number of tickets purchased.
5.To calculate the total booking amount automatically through SQL queries.
6.To retrieve booking information using joins between related tables.
7.To generate reports showing total bookings and movie-wise revenue.
8.To maintain data integrity using primary keys, foreign keys, and constraints.

2.3 Scope

This project is a basic database application intended for academic purposes. It manages sample movie and booking records using MySQL. It does not include online payments, real-time seat selection, or an online user interface.

Within the scope

- Designing four related tables with proper keys and constraints.
- Inserting sample data and retrieving it with SELECT queries and joins.
- Updating records and generating revenue and booking reports.

Outside the scope

- Online payment gateway, seat-map selection, user login and a web or mobile front end.
- Multi-screen or multi-branch cinema management.

3. PBL / CASE STUDY: CINEMA BOOKING MANAGEMENT

Project Based Learning (PBL)approach. In this section the project is treated as a casestudy. A realistic problem of a small cinema is first described, its difficulties are analysed, a database solution is proposed, and the solution is then implemented in the later sections of this file. The cinema described below is a fictional example used only for academic purposes.

3.1 Background of the Case

Consider a small single-screen cinema that shows three popular films in a day, at different timings, at different ticket prices. Customers book tickets at the counter or by phone. The staff write the customer name, film, show time and number of tickets in a paper register, and calculate the amount by hand at the end of the day.

As the number of customers grows, the staff face repeated problems: the same customer is written many times with different spellings, the ticket price changes for weekends but old entries are not corrected, and the owner cannot quickly find out which film earns the most money.

3.2 Analysis of the Existing Manual System

Problem	Effect on the cinema
Data redundancy	Customer and film details are written again and again in every entry, which wastes time and space.
Calculation errors	Amount = tickets x price is calculated by hand, so wrong totals are possible.
Slow searching	Finding all bookings of one customer or one film requires turning pages of the register.
Inconsistent data No integrity checks	The same film or customer may be written in different ways, so reports do not match.
	A booking can be written for a show that was never scheduled.
Poor reporting	Daily revenue and film-wise earnings need long manual addition.

3.3ProposedSystem

The proposed solution is a relational database in which each type of information is stored once in its own table. Movies and customers are stored once; shows refer to movies; bookings refer to customers and shows. Because of these links, the database can combine the data whenever a report is needed.

The booking amount is never typed in. It is calculated by the query as tickets_booked x ticket_price, so it is always correct and automatically changes if the ticket price changes.

3.4 Requirement Analysis

Functional requirements

- Store and display movie details (title, genre, duration).
- Store customer details (name, unique email, phone).
- Schedule shows with date, time, price and available seats for each movie.
- Record bookings linking a customer to a show with the number of tickets.
- Generate reports: booking details, total revenue, bookings per movie, high-value bookings.

Non-functional requirements

- **Integrity:** invalid references and invalid values (negative price, zero tickets) must be rejected.
- **Consistency:** every fact should be stored in only one place.
- **Usability:** reports must be obtainable with short, readable SQL queries.
- **Portability:** the script must run from top to bottom on any MySQL 8 installation.

Hardware and software requirements

Requirement	Details
Processor / RAM	Any modern processor, 4 GB RAM or more
Operating system	Windows
DBMS	MySQL Server 8.x
Client tool	MySQL Workbench 8.x
Language	SQL (Structured Query Language)

3.5Feasibility Study

Type	Assessment
Technical	MySQL and MySQL Workbench are free, widely used and easy to install, so the project is technically feasible.
Economic	No paid software is needed, so the cost of development is almost zero.
Operational	Simple SQL queries can be run by non-technical staff once the reports are saved.
Schedule	The four-table design is small and can be completed within a few days.

3.6 Expected Benefits

- Accuracy: amounts and revenue are calculated by SQL, not by hand.
- Speed: bookings of any customer or film can be found in seconds.
- Reliability: constraints prevent invalid or duplicate records.
- Better decisions: movie-wise revenue shows which film is most profitable.

3.7 Learning Outcomes of this PBL

- Converting a real-life problem into tables, keys and relationships.
- Applying DDL and DML commands in MySQL.
- Writing joins, aggregate functions and filtering conditions to answer business questions.
- Understanding integrity constraints and why they are needed.

4. MODULES AND TOOLS & TECHNOLOGIES USED

4.1 Modules

Module	Function
1. Movie Management	Stores movie ID, title, genre, and duration.
2. Customer Management	Maintains customer ID, name, email, and phone number.
3. Show Management	Stores show ID, movie ID, date, time, ticket price, and available seats.
4. Booking Management	Records booking ID, customer, show, number of tickets, and booking date.
5. Report Generation	Displays booking details, calculates total revenue, and shows the number of bookings for each movie.

4.2 Tools and Technologies

Tool	Use in this project
MySQL	Relational database management system used to store the data
MySQL Workbench	Graphical interface to write and run SQL and see the result grid
SQL	Language used for creating, inserting, updating and querying data
Windows	Operating system used for development
GitHub	Online repository where the project files are uploaded

4.3 Database Concepts Used

- Data DefinitionLanguage(DDL): CREATE DATABASE, CREATE TABLE.
- Data Manipulation Language (DML): INSERT, UPDATE (and DELETE).
- Primary and foreign keys.
- NOT NULL, UNIQUE, and CHECK constraints.
- One-to-many relationships.
- WHERE, ORDER BY, and GROUP BY clauses.
- INNER JOIN and LEFT JOIN.
- Aggregate functions COUNT() and SUM().

5. THEORETICAL BACKGROUND (DBMS CONCEPTS USED)

5.1Database andDBMS

A database is an organised collection of related data. A Database Management System (DBMS) is software that lets users create, store, retrieve, update and protect that data. In a relational DBMS (RDBMS) such as MySQL, the data is arranged in tables made of rows (records or tuples) and columns (attributes or fields).

File system versus DBMS

Point	Manual / filesystem	DBMS (this project)
Redundancy	High, same data repeated	Low, data stored once and linked
Searching	Slow and manual	Fast using SQL queries
Integrity	No automatic checks	Keys and constraints enforce rules
Reports	Calculated by hand	Produced by aggregate queries

5.2 Keys

- **Primary key:** a column (or set of columns) that uniquely identifies each row and cannot be NULL. Here: movie_id, customer_id, show_id and booking_id.
- **Foreign key:** a column that refers to the primary key of another table and maintains referential integrity. Here: Shows.movie_id, Bookings.customer_id and Bookings.show_id.
- **Unique key:** ensures no two rows have the same value, but allows a single NULL. Here: Customers.email.

5.3 Constraints Used

Constraint	Purpose	Where used
PRIMARY KEY	Unique, non-null row identifier	All four tables
FOREIGN KEY	Links tables, blocks invalid	Shows, Bookings
NOT NULL	references	title, customer_name, show_date, show_time, ticket_price ...
UNIQUE	Value must be given	Customers.email
CHECK	No duplicate values	ticket_price > 0, available_seats >= 0, tickets_booked > 0
	Value must satisfy a condition	

5.4 Relationships

A one-to-many(1:N) relationship means one row of the first table can be linked to many rows of the second table, while each row of the second table belongs to only one row of the first.

- One Movie has many Shows (Movies 1 : N Shows).
- One Show has many Bookings (Shows 1 : N Bookings).
- One Customer makes many Bookings (Customers 1 : N Bookings).

Bookings acts as a link between Customers and Shows. Together they represent a many-to-many relationship between customers and shows (a customer can book many shows, and a show can be booked by many customers).

5.5 Normalization

Normalization is the process of organising tables to reduce redundancy and avoid update, insert and delete anomalies. The design of this project follows the first three normal forms:

Normal form	Rule	How the design satisfies it
1NF	Every column holds atomic (single) values; no repeating groups	Each cell holds one value; name, email and phone are separate columns
2NF	1NF and no partial dependency on part of a key	Every table has a single-column primary key, so no partial dependency exists
3NF	2NF and no transitive dependency	Movie details are kept in Movies, not in Shows; customer details are kept in Customers, not in Bookings; the total amount is calculated, not stored

5.6 SQL Command Categories

Category	Commands	Used in this project for
DDL	CREATE, ALTER, DROP	Creating the database and four tables
DML	INSERT, UPDATE, DELETE	Adding sample data and changing a ticket price
DQL	SELECT	All reports and queries
TCL / DCL	COMMIT, ROLLBACK / GRANT, REVOKE	Not required in this mini project

5.7 Joins and Aggregate Functions

- **INNER JOIN:** returns only the rows that have matching values in both tables.
- **LEFT JOIN:** returns all rows of the left table and the matching rows of the right table; unmatched rows get NULL. It is used to find movies with no bookings.
- **SUM():** adds the values of a numeric column or expression, for example total revenue.
- **COUNT():** counts rows, for example the number of bookings per movie.
- **GROUP BY:** groups rows so that an aggregate function is applied to each group.

5.8 Revenue Formula

Booking amount = tickets_booked x ticket_price and **Total revenue = SUM(Booking amount)**

6. DATABASE DESIGN

6.1 Entity Relationship (ER) Diagram

Thediagram belowshows thefour tables,their primary keys (PK), foreign keys (FK) and the one-to-many relationships between them.



Figure 1: ER diagram of the Movie Ticket Booking System

6.2 Table Structure

Table	Primary key	Foreign key
Movies	movie_id	None
Customers	customer_id	None
Shows	show_id	movie_id references Movies
Bookings	booking_id	customer_id references Customers; show_id references Shows

Important: A primary key uniquely identifies each row. A foreign key connects one table to another and helps maintain referential integrity.

6.3 Data Dictionary

Table: Movies

Column	Datatype	Key	Constraint
movie_id	INT	PK	Unique, not null
title	VARCHAR(100)		NOT NULL
genre	VARCHAR(50)		-
duration	INT		Minutes

Table: Customers

Column	Data type	Key	Constraint
customer_id	INT	PK	Unique, not null
customer_name	VARCHAR(100)		NOT NULL
email	VARCHAR(100)		UNIQUE
phone	VARCHAR(15)		-

Table: Shows

Column	Data type	Key	Constraint
show_id	INT	PK	Unique, not null
movie_id	INT	FK	NOT NULL, references Movies
show_date	DATE		NOT NULL
show_time	TIME		NOT NULL
ticket_price	DECIMAL(8,2)		NOT NULL, CHECK > 0
available_seats	INT		NOT NULL, CHECK >= 0

Table: Bookings

Column	Data type	Key	Constraint
booking_id	INT	PK	Unique, not null
customer_id	INT	FK	NOT NULL, references Customers
show_id	INT	FK	NOT NULL, references Shows
tickets_booked	INT		NOT NULL, CHECK > 0
booking_date	DATE		NOT NULL

7. IMPLEMENTATION: DATABASE, TABLES AND SAMPLE DATA

Open MySQL Workbench, connectto thelocal MySQLserver,click theNewSQLTabicon,

and execute the statements in the order shown. Each figure shows the SQL statement and its Action Output.

7.1 Creating the Database

CREATE DATABASE creates the database and USE selects it, so that all following tables are created inside movie_ticket_booking.

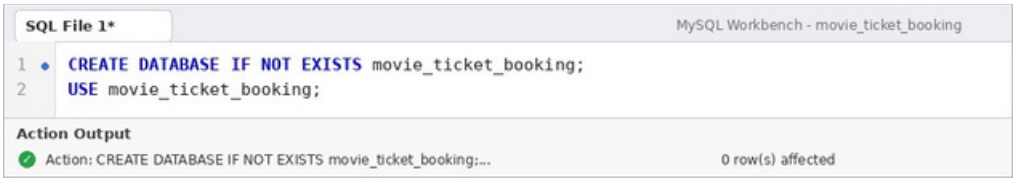


Figure2: Database creation and selection

7.2 Creating the Tables (DDL)

INT stores whole numbers, VARCHAR stores text, DATE stores dates, TIME stores times, and DECIMAL stores prices. The tables are created in the order Movies, Customers, Shows, Bookings because a table with a foreign key can be created only after the table it refers to exists.



Figure 3: CREATE TABLE Movies

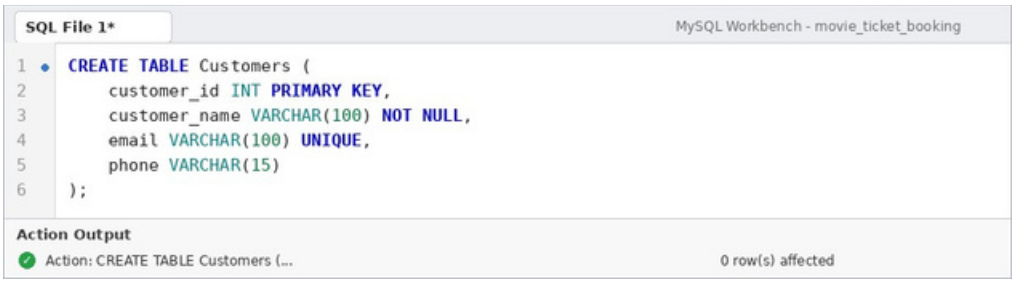


Figure 4: CREATE TABLE Customers (with UNIQUE email)

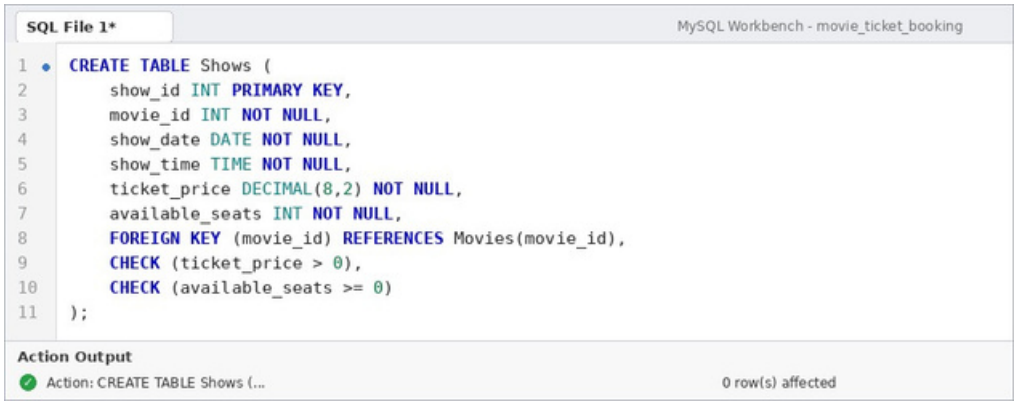


Figure 5: CREATE TABLE Shows (foreign key and CHECK constraints)



Figure 6: CREATE TABLE Bookings (two foreign keys)

The foreign keys ensure that a show cannotreference a nonexistentmovie and a booking cannot reference a nonexistent customer or show.

7.3 Inserting Sample Data (DML)

The following records are sample data for demonstration. They do not represent actual cinema bookings. Parent tables (Movies, Customers) are filled first, then Shows, and finally Bookings, so that every foreign key value already exists.

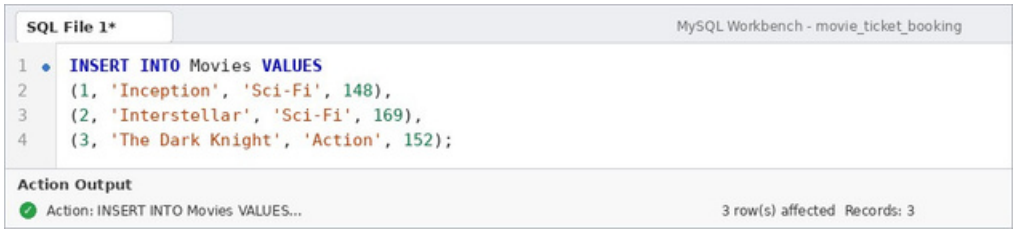


Figure 7: INSERT into Movies



Figure 8: INSERT into Customers

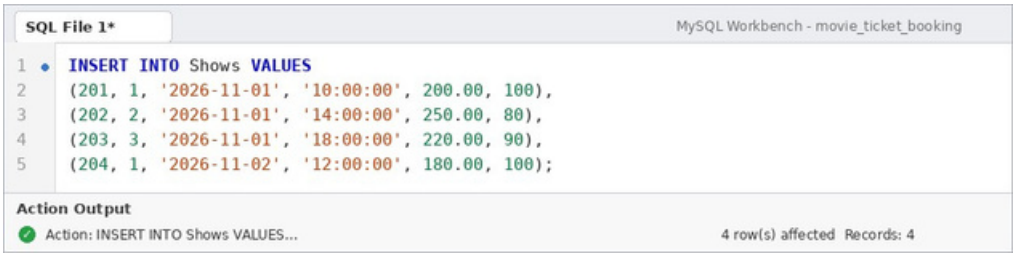


Figure 9: INSERT into Shows

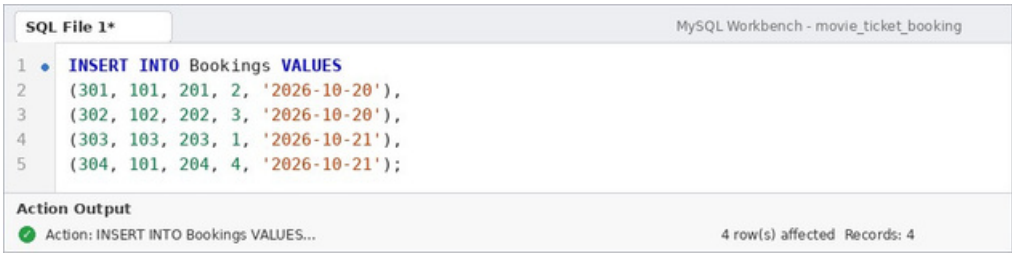


Figure 10: INSERT into Bookings

8. SQL QUERIES WITH OUTPUT SCREENSHOTS

These queries are the most important part of the practical work. Each query was run separately and the result grid is shown below the statement. Queries 1 to 10 are run on the original sample data; the UPDATE in Query 11 is run last so that the earlier outputs match the sample data.

Query 1: Display all movies

Purpose: To list every movie stored in the database.

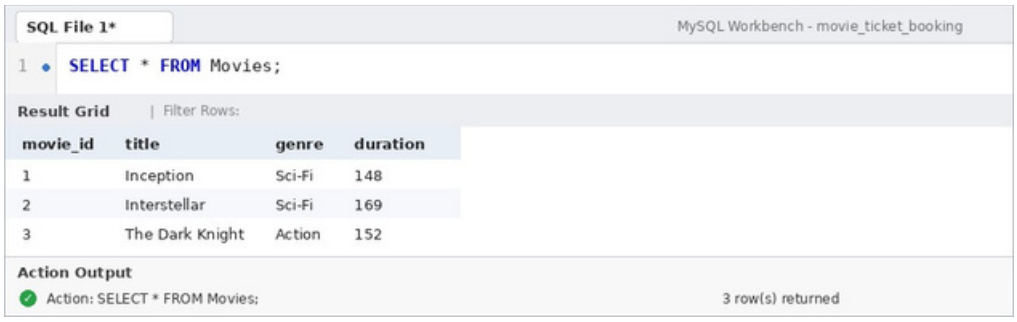


*Figure11:SELECT*FROMMovies*

Explanation: The asterisk selects all columns. Three movies are returned.

Query 2: Display all customers

Purpose: To list all registered customers.

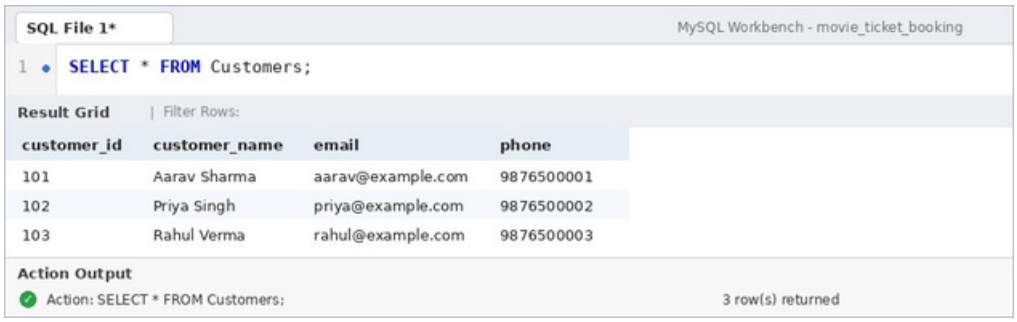


*Figure12:SELECT*FROMCustomers*

Query3: Displayshows withmovie titles (INNERJOIN)

Purpose: To show each show along with the title of its movie.

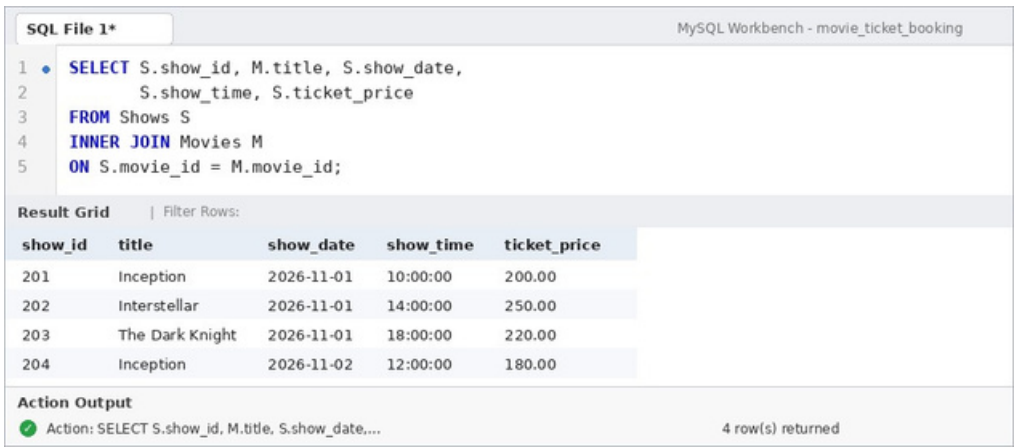


Figure 13: INNER JOIN of Shows and Movies

Explanation: The Shows table stores only movie_id. The join on S.movie_id = M.movie_id replaces it with the readable title.

Query 4: Display complete booking details (four-table join)

Purpose: To combine all four tables and calculate the amount of each booking.

SQL File 1* — MySQL Workbench - movie_ticket_booking

```
SELECT B.booking_id, C.customer_name,
       M.title, B.tickets_booked,
       S.ticket_price,
       B.tickets_booked * S.ticket_price AS total_amount
FROM Bookings B
INNER JOIN Customers C
    ON B.customer_id = C.customer_id
INNER JOIN Shows S
    ON B.show_id = S.show_id
INNER JOIN Movies M
    ON S.movie_id = M.movie_id
ORDER BY B.booking_id;
```

Result Grid | Filter Rows:

booking_id	customer_name	title	tickets_booked	ticket_price	total_amount
301	Aarav Sharma	Inception	2	200.00	400.00
302	Priya Singh	Interstellar	3	250.00	750.00
303	Rahul Verma	The Dark Knight	1	220.00	220.00
304	Aarav Sharma	Inception	4	180.00	720.00

Action Output

Action: SELECT B.booking_id, C.customer_name,... 4 row(s) returned

Figure14: Bookingdetails withcalculatedtotal_amount

Explanation: Bookings is joined with Customers, Shows and Movies. The expression tickets_booked * ticket_price gives total_amount, for example 2 x 200.00 = 400.00.

Query 5: Calculate total revenue (SUM)

Purpose: To find the total money earned from all bookings.

SQL File 1* — MySQL Workbench - movie_ticket_booking

```
SELECT SUM(B.tickets_booked * S.ticket_price)
       AS total_revenue
FROM Bookings B
INNER JOIN Shows S
ON B.show_id = S.show_id;
```

Result Grid | Filter Rows:

total_revenue
2090.00

Action Output

Action: SELECT SUM(B.tickets_booked * S.ticket_price)... 1 row(s) returned

Figure 15: Total revenue using SUM()

Explanation: SUM adds all booking amounts: 400 + 750 + 220 + 720 = 2090.00.

Query 6: Count bookings for each movie (COUNT and GROUP BY)

Purpose: Tofindhow manybookingseachmovie hasreceived.

SQL File 1* — MySQL Workbench - movie_ticket_booking

```
SELECT M.title,
       COUNT(B.booking_id) AS total_bookings
FROM Movies M
INNER JOIN Shows S ON M.movie_id = S.movie_id
INNER JOIN Bookings B ON S.show_id = B.show_id
GROUP BY M.movie_id, M.title;
```

Result Grid | Filter Rows:

title	total_bookings
Inception	2
Interstellar	1
The Dark Knight	1

Action Output

Action: SELECT M.title,... 3 row(s) returned

Figure 16: Movie-wise booking count

Explanation: GROUP BY creates one group per movie and COUNT counts the bookings in each group. Inception has 2 because it has two shows with one booking each.

Query 7: Display bookings above Rs. 500 (WHERE)

Purpose: To filter high-value bookings.

SQL File 1* — MySQL Workbench - movie_ticket_booking

```
SELECT B.booking_id, C.customer_name,
       B.tickets_booked * S.ticket_price AS amount
FROM Bookings B
INNER JOIN Customers C
ON B.customer_id = C.customer_id
INNER JOIN Shows S
ON B.show_id = S.show_id
WHERE B.tickets_booked * S.ticket_price > 500;
```

Result Grid | Filter Rows:

booking_id	customer_name	amount
302	Priya Singh	750.00
304	Aarav Sharma	720.00

Action Output

Action: SELECT B.booking_id, C.customer_name,... 2 row(s) returned

Figure 17: Bookings with amount greater than 500

Explanation: The WHERE clause keeps only rows whose calculated amount is above 500. Bookings 302 and 304 qualify.

Query 8: Movie-wise revenue (additional report)

Purpose: Tofindhow mucheachmovie earned.

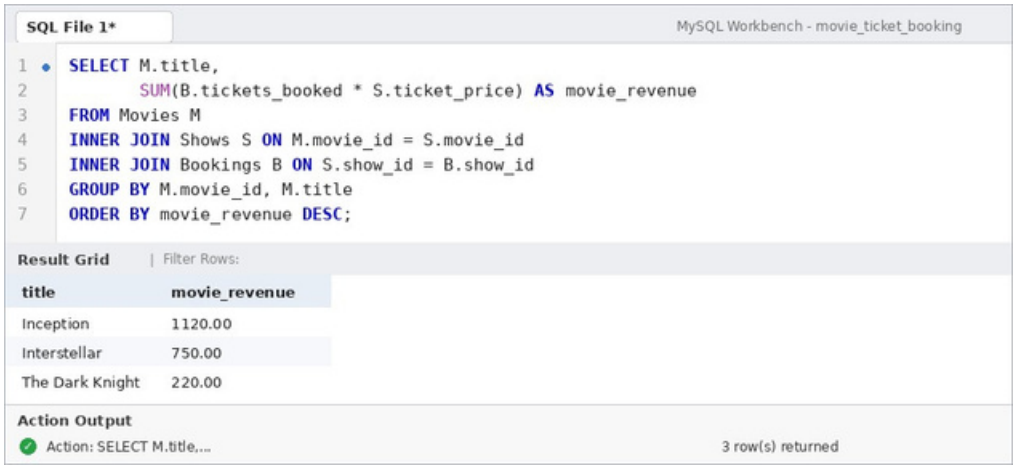


Figure 18: Revenue per movie using SUM() and GROUP BY

Explanation: Inception earns 400 + 720 = 1120.00 and is the highest earning film. ORDER BY ... DESC lists the highest revenue first.

Query 9: Display movies without any bookings (LEFT JOIN)

Purpose: To find movies that nobody has booked yet.



Figure 19: LEFT JOIN with IS NULL (empty result)

Explanation: LEFT JOIN keeps every movie; if no booking exists, B.booking_id is NULL. All three movies have bookings, so the result is empty. An empty result is expected and is a valid demonstration of the query.

Query 10: Display the total number of bookings

Purpose: To count all rows of the Bookings table.

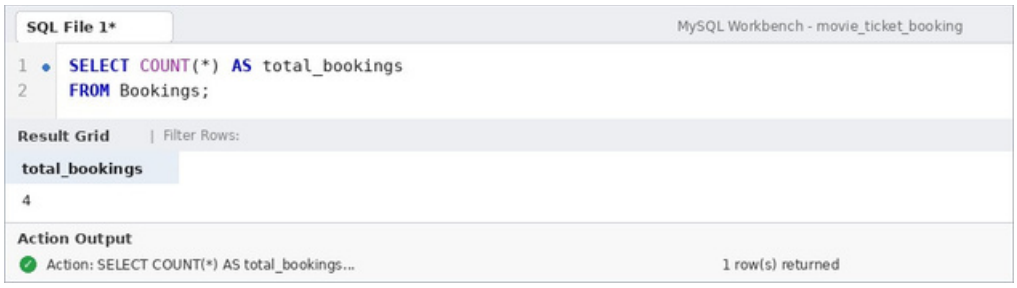


Figure 20: Total bookings using COUNT(*)

Query 11: Update a ticket price (UPDATE)

Purpose: Tochangetheticketpriceofshow201from 200.00 to 210.00.

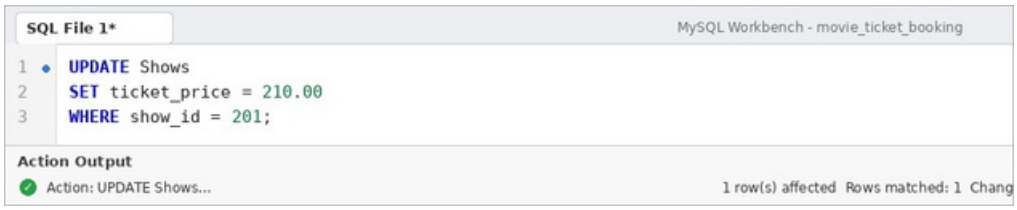


Figure 21: UPDATE statement on the Shows table

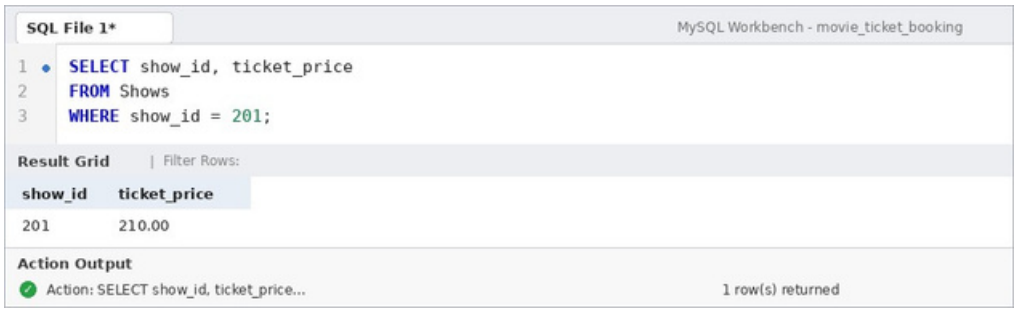


Figure 22: SELECT to verify the updated price

Explanation: The WHERE clause restricts the change to show 201 only. Without WHERE, every show would get the new price. After the update, bookings of show 201 are calculated with 210.00 automatically, because the amount is calculated and not stored.

9. FINAL VERIFICATION

Finally, the database is checked to confirm that all four tables exist and that each contains the expected number of records.

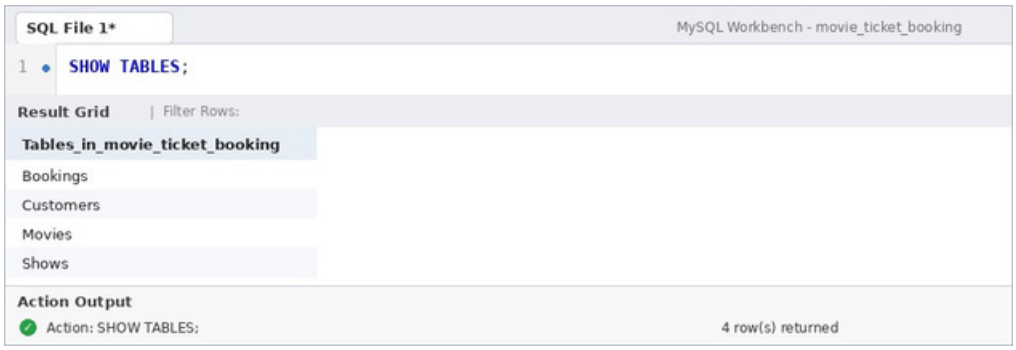


Figure 23: SHOW TABLES: all four tables are present



Figure 24: Record counts: 3 movies, 3 customers, 4 shows, 4 bookings

Table	Expected records	Result
Movies	3	Verified
Customers	3	Verified
Shows	4	Verified
Bookings	4	Verified

10. RESULTS AND DISCUSSION

AllSQL statements wereexecutedsuccessfullyand returned the expected results. The important resultsare summarised below.

Report	Result
Total revenue (before update)	Rs. 2,090.00
Total bookings	4
Highest booking amount	Rs. 750.00 (Booking 302, Interstellar)
Highest earning movie	Inception, Rs. 1,120.00
Movies with no bookings	None (empty result)
Bookings above Rs. 500	2 (Bookings 302 and 304)

10.1Discussion

- The joins confirm that the relationships between the tables work: a booking can be traced to a customer, a show and a movie.
- The calculated amount stays correct after the price update because it is derived from ticket_price and not stored.
- Foreign key and CHECK constraints protect the data: a booking for a non-existing show, a zero ticket count or a negative price is rejected by MySQL.
- The report of movie-wise revenue gives the cinema owner useful information, for example which film attracts the most customers.

10.2HowthePBLProblem WasSolved

Problem in manual system	Solution in the database
Data redundancy	Separate tables linked by keys; data stored once
Calculation errors	Amount calculated by SQL expression
Slow searching	SELECT with WHERE and JOIN
No integrity checks	Primary key, foreign key, CHECK and UNIQUE constraints
Poor reporting	SUM, COUNT and GROUP BY reports

11. LIMITATIONS AND FUTURE SCOPE

11.1 Limitations

- The project uses a small set of sample data.
- There is no graphical front end; all work is done with SQL queries.

- Seat numbers are not stored, so two customers cannot be prevented from booking the same seat.
- The available_seats value is not reduced automatically when a booking is added.
- Online payment and user login are not included.

11.2 Future Scope

- Add a Seats table to support seat selection and avoid double booking.
- Use a trigger to reduce available_seats after every booking.
- Add a Theatres / Screens table for a multi-screen cinema.
- Create views and stored procedures for daily revenue and booking reports.
- Build a web or mobile front end (for example with PHP, Python or Java) connected to this database.
- Add payment details, cancellation and refund records.

12. CONCLUSION

The Movie Ticket Booking System was successfully designed using MySQL. The project demonstrates how a relational database can be used to store and manage movie information, customer details, show schedules, and ticket booking records.

The project implements primary keys, foreign keys, constraints, data insertion, data retrieval, data updating, joins, and aggregate functions. It also calculates ticket booking amounts and generates simple reports such as total revenue and movie-wise booking counts.

The case study showed how the problems of a manual register, such as repeated data, calculation mistakes and slow searching, can be solved by a well-designed database.

Through this project, I gained practical knowledge of database design, relationships between tables, SQL queries, and data integrity. The system provides a basic foundation for developing a more advanced movie ticket booking application in the future.

13. VIVA PREPARATION: QUESTIONS AND ANSWERS

Question	Answer
Why did you use primary keys?	A primary key uniquely identifies every row, so records can be found without confusion and duplicates are avoided.
How are the tables connected?	Through foreign keys: Shows.movie_id refers to Movies, and Bookings.customer_id and Bookings.show_id refer to Customers and Shows.
How is revenue calculated?	Each booking amount is tickets_booked x ticket_price. SUM() of these amounts gives the total revenue (Rs. 2,090.00).
Why is the total amount not stored?	It can be derived from other columns. Storing it would create redundancy and it could become wrong when the price changes (3NF).
What is the difference between INNER JOIN and LEFT JOIN?	INNER JOIN returns only matching rows; LEFT JOIN returns all rows of the left table with NULL where no match exists.
What is referential integrity?	A rule that a foreign key value must exist in the referenced table, so no booking can point to a missing customer or show.
What is the use of GROUP BY? What does the CHECK constraint do?	It groups rows with the same value so that COUNT or SUM is calculated for each group, for example per movie.
	It rejects values that break a condition, for example ticket_price <= 0 or tickets_booked <= 0.
Why is the update query run last?	Because it changes the data; running it last keeps the earlier outputs equal to the original sample data.
Which normal form is the design in?	Third Normal Form (3NF): atomic values, single-column keys, andno transitive dependencies.

14. REFERENCES AND GITHUB REPOSITORY

14.1References

1.MySQL 8.0 Reference Manual, Oracle Corporation: https://dev.mysql.com/doc/refman/8.0/en/

2.MySQL Workbench Manual: https://dev.mysql.com/doc/workbench/en/

3.Ramez Elmasri and Shamkant B. Navathe, Fundamentals of Database Systems, Pearson.

4.Abraham Silberschatz, Henry F. Korth and S. Sudarshan, Database System Concepts, McGraw-Hill.

5.W3Schools SQL Tutorial: https://www.w3schools.com/sql/

14.2 GitHub Repository

The complete SQL script, README file and screenshots are uploaded in the following repository:

Item	Details
Repository name	movie-ticket-booking-system
Repository URL	https://github.com/_____/movie-ticket-booking-system
Files	movie_ticket_booking.sql, README.md, screenshots/, project_report.pdf

Signature of Student: _____

Signature of Teacher: _____